



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Computação Gráfica

Trabalho Prático

Grupo 15

Edgar Ferreira	Fernando Pires	Sara Silva	Zita Duarte
a99890	a77399	a104608	a104268

Data da Receção	
Responsável	
Avaliação	
Observações	

Computação Gráfica

Edgar Ferreira
a99890

Fernando Pires
a77399

Sara Silva
a104608

Zita Duarte
a104268

abril, 2025

Índice

1. Introdução	1
2. Generator	2
2.1. Superfícies de Bézier	2
2.1.1. Leitura dos ficheiros .patch	2
2.1.2. Geração dos triângulos	3
2.1.3. Escrita para ficheiro .3d	4
3. Engine	6
3.1. VBOs, IBOs e VAOs	6
3.1.1. Modelos	6
3.1.2. Preparação dos modelos	6
3.1.3. Desenho dos vértices	7
3.1.4. Benefícios	8
3.2. Animações Temporizadas	8
3.2.1. Transformações	8
3.2.1.1. Translação	9
3.2.1.2. Rotação	11
4. Sistema Solar	12
5. Testes	13
6. Conclusões	14

Lista de Figuras

Figura 1	Representação do cometa gerado através do ficheiro <code>comet.patch</code>	5
Figura 2	Representação da cena do Sistema Solar com rotações e translações animadas, incluindo o cometa	12
Figura 3	Teste 1 - Rotação animada de um <i>teapot</i> animado a seguir uma curva de Catmull-Rom com alinhamento ativado	13
Figura 4	Teste 2 - <i>Teapot</i> estático	13

1. Introdução

Este relatório foi realizado no âmbito da Unidade Curricular de **Computação Gráfica** (CG) no ano letivo 2024/2025 e descreve o desenvolvimento da **terceira fase do projeto**. Nesta fase, o objetivo foi alterar o **Generator** para criar as **superfícies de Bézier**, enquanto no **Engine** o foco foi evoluir o projeto para suportar o uso de **VBOs** e animações temporizadas com recurso às **curvas de Catmull-Rom**.

2. Generator

2.1. Superfícies de Bézier

Nesta fase, introduzimos um novo tipo de geração de modelos 3D, com base em **superfícies de Bézier bicúbicas**, um método clássico que nos permite criar novas superfícies suaves.

2.1.1. Leitura dos ficheiros .patch

Os ficheiros de leitura `.patch` fornecidos apresentam a seguinte estrutura:

- Número de *patches*;
- Índices dos pontos de controlo de cada *patch* (16 por *patch*);
- Pontos de controlo;

Inicialmente, é feita uma leitura do ficheiro:

1. Abrir o ficheiro e ler o número de *patches*.

```
if (!file.is_open()) {
    std::cerr << "Erro ao abrir ficheiro de patch: " << patchFile << std::endl;
    return result;
}

size_t numPatches;
file >> numPatches;
```

2. Ler os índices de 16 pontos de controlo para cada *patch* e armazená-los numa estrutura de vetores.

```
std::vector<std::vector<size_t>> patches(numPatches, std::vector<size_t>(16));

for (size_t i = 0; i < numPatches; ++i) {
    for (size_t j = 0; j < 16; ++j) {
        size_t idx;
        file >> idx;
        file.ignore();
        patches[i][j] = idx;
    }
}
```

3. Ler os **pontos de controlo** propriamente ditos, armazenando cada tripla de coordenadas (x,y,z) num vetor de `Point`.

```
size_t numPoints;
file >> numPoints;
std::vector<Point> controlPoints;
for (size_t i = 0; i < numPoints; ++i) {
    float x, y, z;
    file >> x;
    file.ignore();
    file >> y;
    file.ignore();
    file >> z;
    file.ignore();
    controlPoints.push_back(Point(x, y, z));
}
```

2.1.2. Geração dos triângulos

Uma vez lidos os *patches* e os respetivos pontos de controlo, é necessário calcular os pontos que pertencem à **superfície de Bézier bicúbica**. Cada *patch* representa uma superfície definida por uma grelha 4x4 de pontos de controlo, e a sua avaliação é feita através da interpolação dos parâmetros *u* e *v*, ambos no intervalo $[0, 1]$.

A fórmula matemática utilizada para calcular um ponto na superfície é:

$$P(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u) \cdot B_j(v) \cdot P_{ij}$$

onde:

- $B_i(t)$ são os **coeficientes derivados do binómio de Newton**, calculados na função:

```
float binomial(float t, float* b) {
    float it = 1 - t;
    b[0] = it * it * it;
    b[1] = 3 * t * it * it;
    b[2] = 3 * t * t * it;
    b[3] = t * t * t;
}
```

- Já P_{ij} representa os pontos de controlo, armazenados em `controlPoints[4][4][3]`, que são utilizados para calcular o ponto resultante:

```

Point bezierPoint(float u, float v, float controlPoints[4][4][3]) {
    Point p(0, 0, 0);
    float bu[4], bv[4];
    binomial(u, bu);
    binomial(v, bv);
    for (int i = 0; i < 4; ++i) {
        for (int j = 0; j < 4; ++j) {
            float b = bu[i] * bv[j];
            p.x += controlPoints[i][j][0] * b;
            p.y += controlPoints[i][j][1] * b;
            p.z += controlPoints[i][j][2] * b;
        }
    }
    return p;
}

```

Com base no nível de tesselação fornecida, cada *patch* será dividido em pequenos retângulos, compostos por dois triângulos. Os seus vértices são calculados através da função anterior `bezierPoint` e posteriormente serão armazenados em no vetor de pontos `result` :

```

for (int i = 0; i < tessellation; i++) {
    for (int j = 0; j < tessellation; j++) {
        float u = (float)i / tessellation;
        float v = (float)j / tessellation;
        float u1 = (float)(i + 1) / tessellation;
        float v1 = (float)(j + 1) / tessellation;

        Point p1 = bezierPoint(u, v, patchPoints);
        Point p2 = bezierPoint(u1, v, patchPoints);
        Point p3 = bezierPoint(u, v1, patchPoints);
        Point p4 = bezierPoint(u1, v1, patchPoints);

        result.push_back(p1);
        result.push_back(p3);
        result.push_back(p2);

        result.push_back(p2);
        result.push_back(p3);
        result.push_back(p4);
    }
}

```

2.1.3. Escrita para ficheiro .3d

Depois de gerar os triângulos, os vértices calculados e armazenados num vetor são passados para a função referida nas fases anteriores `saveToFile`, que guarda os pontos no ficheiro `.3d` pretendido.

Como exemplo e teste desta funcionalidade, temos o cometa usado na cena do sistema solar:

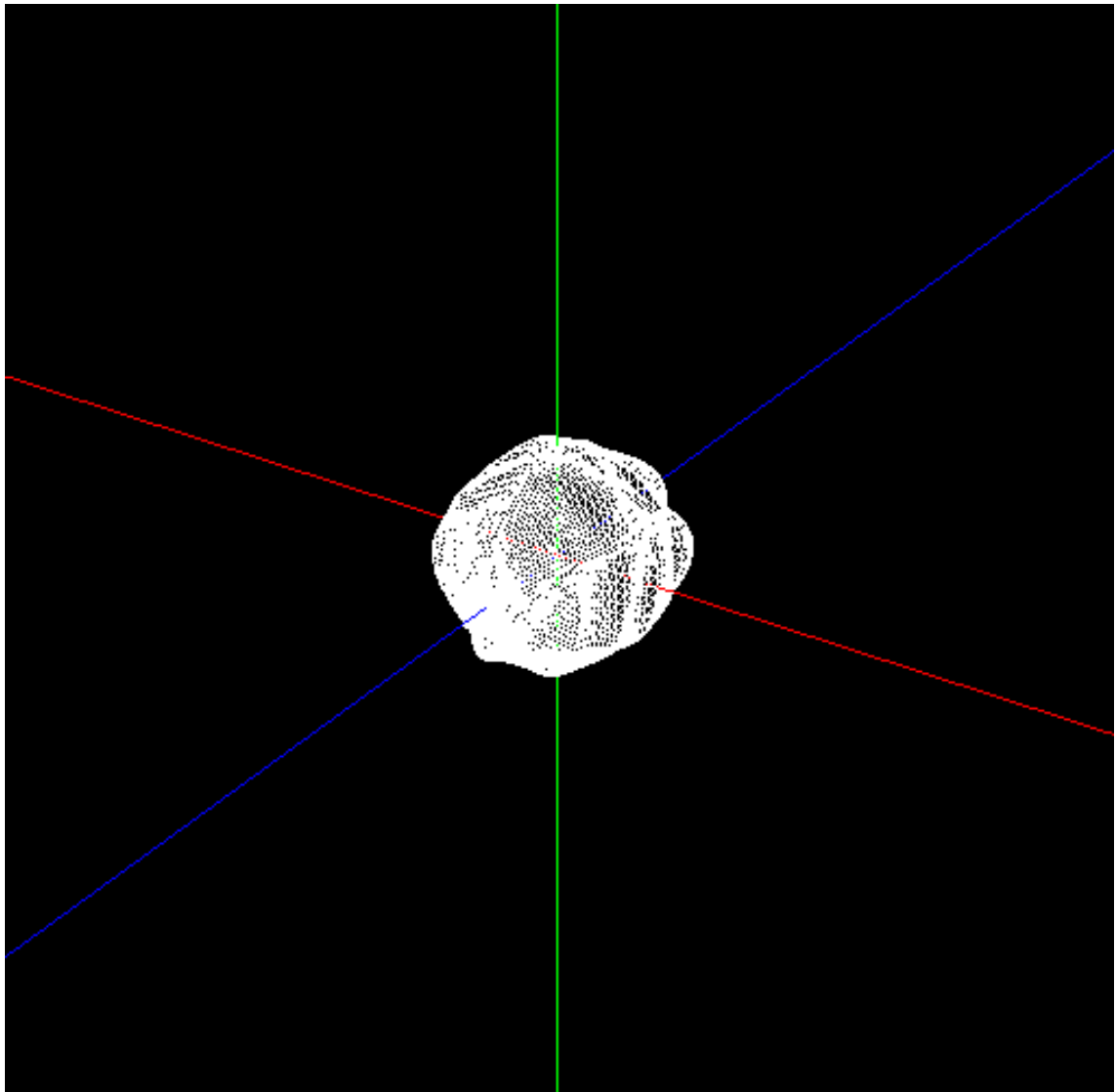


Figura 1: Representação do cometa gerado através do ficheiro `comet.patch`

3. Engine

3.1. VBOs, IBOs e VAOs

Com a terceira fase do projeto, uma das mudanças estruturais mais relevantes foi a substituição da renderização imediata por um pipeline moderno baseado em **Vertex Buffer Objects (VBOs)**, **Index Buffer Objects (IBOs)** e **Vertex Array Objects (VAOs)**.

3.1.1. Modelos

Esta transição implicou alterações na classe `Model`, criada em fases anteriores. Para além da lista original de pontos, cada modelo passou a manter:

- Um vetor `vertices` com os vértices únicos (não duplicados);
- Um vetor `indices` que define a ordem de desenho dos triângulos com base em `vertices`;
- Três identificadores OpenGL: `vao`, `vbo`, e `ebo`, respetivamente para o objeto de vértice, buffer de vértices e buffer de índices.

```
class Model {
public:
    Model();
    Model(const std::vector<Point>& points);

    void draw();
    void prepareModel();

private:
    std::vector<Point> rawPoints;
    std::vector<Point> vertices;
    std::vector<unsigned int> indices;
    GLuint vao, vbo, ebo;
};
```

3.1.2. Preparação dos modelos

Após a leitura das configurações, grupos e seus modelos, e antes da primeira renderização, é chamada a função `prepareModel()` para cada modelo de cada grupo. Esta função é responsável por organizar e carregar os dados na GPU:

```

void Model::prepareModel() {
    std::unordered_map<std::string, unsigned int> uniquePoints;
    vertices.clear();
    indices.clear();

    for (const Point& p : rawPoints) {
        std::string key = std::to_string(p.x) + "," + std::to_string(p.y) + "," +
std::to_string(p.z);
        if (uniquePoints.count(key) == 0) {
            uniquePoints[key] = vertices.size();
            vertices.push_back(p);
        }
        indices.push_back(uniquePoints[key]);
    }

    glGenVertexArrays(1, &vao);
    glGenBuffers(1, &vbo);
    glGenBuffers(1, &ebo);

    glBindVertexArray(vao);

    glBindBuffer(GL_ARRAY_BUFFER, vbo);
    glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(Point), vertices.data(),
GL_STATIC_DRAW);

    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(Point), (void*)0);
    glEnableVertexAttribArray(0);

    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.size() * sizeof(unsigned int),
indices.data(), GL_STATIC_DRAW);

    glBindVertexArray(0);
}

```

Este processo pode ser dividido em quatro etapas principais:

1. **Otimização de vértices:** São eliminadas repetições dos pontos lidos (`rawPoints`) para criar um vetor `vertices` compacto. Os índices associados a cada ponto único são guardados em `indices` .
2. **Criação dos buffers:** São gerados três identificadores: o **VAO** para encapsular toda a configuração, o **VBO** para armazenar os vértices, e o **EBO/IBO** para guardar os índices.
3. **Configuração do pipeline:** Ao associar os buffers ao **VAO**, toda a configuração de atributos fica gravada e pode ser reutilizada com um simples `glBindVertexArray()` , aumentando a eficiência e facilidade.
4. **Envio de dados à GPU:** Os dados são enviados uma única vez para a GPU através do `glBufferData` , e a posterior renderização torna-se extremamente eficiente.

3.1.3. Desenho dos vértices

A função `draw()` é responsável por renderizar o modelo já preparado:

```

void Model::draw() {
    glBindVertexArray(vao);
    glDrawElements(GL_TRIANGLES, indices.size(), GL_UNSIGNED_INT, 0);
    glBindVertexArray(0);
}

```

A chamada a `glBindVertexArray(vao)` ativa automaticamente toda a configuração previamente feita (buffers e atributos), e a chamada a `glDrawElements` desenha os triângulos usando os índices do *EBO*.

Este método é altamente eficiente, pois evita chamadas repetitivas de configuração a cada frame — tudo o que é necessário já está encapsulado no *VAO*.

3.1.4. Benefícios

Antes desta transição, os vértices eram enviados para a GPU a cada frame usando chamadas diretas como `glBegin(GL_TRIANGLES)` e `glVertex3f(...)`, o que resultava em baixa eficiência e maior carga sobre o processador. Além disso, não havia reutilização de dados, tornando a renderização de cenas complexas mais pesadas e menos escalável.

Com os **VBOs** e **IBOs**, os vértices e os índices passam a ser enviados uma única vez para a GPU e armazenados na memória gráfica. O **VAO**, por sua vez, guarda a configuração completa desses buffers, permitindo que toda a estrutura de um modelo seja ativada com uma única chamada `glBindVertexArray()`. Esta abordagem permite um ganho de desempenho significativo, especialmente quando há múltiplos modelos em cena ou animações em tempo real.

3.2. Animações Temporizadas

Outro objetivo desta fase foi a implementação de **animações temporizadas**, que permitam simular um movimento contínuo e rotacional dos modelos ao longo do tempo, criando cenas dinâmicas e mais realistas.

Para tal, introduzimos dois tipos de transformação animada:

- **Translação com curvas de Catmull-Rom**, que descreve trajetórias suaves com base em pontos de controlo e num determinado número de segundos;
- **Rotação temporal**, onde o modelo realiza uma rotação completa num determinado número de segundos.

Estas transformações são descritas diretamente no ficheiro XML da cena, através de novas variantes das instruções `<translate>` e `<rotate>`.

3.2.1. Transformações

Para suportar as novas funcionalidades e estruturas XML, alteramos a estrutura da classe `Transform`. Agora, para além das transformações clássicas (`translate`, `rotate`, `scale`), cada transformação pode conter:

- Um campo `time` para o tempo da animação, no caso de translações e rotações animadas.
- Um campo booleano `align` que determina se o objeto está alinhado com a curva, no caso de translações animadas.
- Um vetor de `Point` que representa os pontos de controlo da curva, no caso de translações animadas;

```

class Transform {
public:
    std::string type;

    float translate[3] = {0, 0, 0};
    float scale[3] = {1, 1, 1};
    float rotate[4] = {0, 0, 0, 0};

    float time = 0;
    bool align = false;
    std::vector<Point> curvePoints;

    Transform() {}
};

```

De forma a suportar esta nova estrutura, a função `parseTransform` teve de ser também alterada.

3.2.1.1. Translação

Uma das transformações adicionadas foi a **translação com curvas de Catmull-Rom**. Esta técnica permite definir o percurso de um modelo com base num conjunto de pontos de controlo, escritos no ficheiro XML. Além do percurso, é também possível especificar se o modelo se deve alinhar com a direção da curva, criando uma animação mais realista — como, por exemplo, um cometa a seguir uma trajetória orbital com orientação constante.

A informação no XML para construção desta translação é estruturada da seguinte forma:

```

...
<translate time="10" align="True" >
  <point x="1" y="0" z="1" />
  <point x="0.707" y="0.707" z="1" />
  <point x="0" y="1" z="1" />
  ...
  <point x="-1" y="0" z="1" />
</translate>
...

```

Após validação de que é uma translação animada com os campos de `time` e pelo menos de quatro pontos de controlo (lista de `point`), a animação é processada pela função `getGlobalCatmullRomPoint`, que calcula a posição e a derivada da curva num instante `t` normalizado, com base no tempo decorrido desde o início da aplicação:

```

float elapsed = glutGet(GLUT_ELAPSED_TIME) / 1000.0f;
float t = fmod(elapsed, transform.time) / transform.time;
getGlobalCatmullRomPoint(t, transform.curvePoints, pos, deriv);

```

Este tempo $t \in [0, 1]$ representa a posição percentual ao longo da curva. A função `getGlobalCatmullRomPoint` seleciona os quatro pontos de controlo relevantes e delega a interpolação à função `getCatmullRomPoint`, responsável por calcular a posição (`pos`) e a derivada (`deriv`) naquele ponto da curva.

```

void getGlobalCatmullRomPoint(float gt, const std::vector<Point>& points, float
*pos, float *deriv) {
    int n = points.size();
    float t = gt * n;
    int index = floor(t);
    t = t - index;

    int indices[4];
    indices[0] = (index + n-1)%n;
    indices[1] = (indices[0]+1)%n;
    indices[2] = (indices[1]+1)%n;
    indices[3] = (indices[2]+1)%n;

    Point p0 = points[indices[0]];
    Point p1 = points[indices[1]];
    Point p2 = points[indices[2]];
    Point p3 = points[indices[3]];

    getCatmullRomPoint(t, p0, p1, p2, p3, pos, deriv);
}

```

A função `getCatmullRomPoint` é o núcleo da **interpolação cúbica da Catmull-Rom**. Para cada coordenada (x, y, z) , a posição é obtida multiplicando:

- A **matriz base** de Catmull-Rom, que define a forma da curva;
- O vetor com os quatro pontos de controle;
- O vetor $T = [t^3, t^2, t, 1]$, que avalia o polinómio de grau 3.

A derivada (direção da curva) é obtida da mesma forma, usando o vetor $T' = [3t^2, 2t, 1, 0]$.

```

float M[4][4] = {
    {-0.5f, 1.5f, -1.5f, 0.5f},
    { 1.0f, -2.5f, 2.0f, -0.5f},
    {-0.5f, 0.0f, 0.5f, 0.0f},
    { 0.0f, 1.0f, 0.0f, 0.0f}
};

float T[4] = {t*t*t, t*t, t, 1};
float Tderiv[4] = {3*t*t, 2*t, 1, 0};

```

A seguir, para cada coordenada, o cálculo é feito com:

```

float px[4] = {p0.x, p1.x, p2.x, p3.x};
// Repetido para py[] e pz[]
multMatrixVector(M, px, ax); // ax = coeficientes do polinómio da coordenada x
// ...
pos[0] = T[0]*ax[0] + T[1]*ax[1] + T[2]*ax[2] + T[3]*ax[3];
deriv[0] = Tderiv[0]*ax[0] + Tderiv[1]*ax[1] + Tderiv[2]*ax[2] + Tderiv[3]*ax[3];

```

Com isto, temos tanto a **posição atual** ao longo da curva (`pos`), como a **direção instantânea do movimento** (`deriv`) — isto é, a derivada da curva naquele ponto.

Retomando à função de desenho e aplicação da transformação, se a opção `align` for igual a `true`, o objeto será orientado para coincidir com a direção da curva. Este alinhamento é feito através da construção de uma matriz de rotação a partir da derivada calculada, de produtos vetoriais (`cross`) e de normalizações dos vetores (`normalize`).

```

if (transform.align) {
    float up[3] = {0, 1, 0};
    float right[3];

    cross(deriv, up, right);
    normalize(right);

    cross(right, deriv, up);
    normalize(up);

    float m[16];
    buildRotMatrix(deriv, up, right, m);
    glMultMatrixf(m);
}

```

Os três vetores usados (`deriv` , `up` e `right`) formam uma **base ortonormal**. Com ela e chamando a função `buildRotMatrix` , constrói-se uma **matriz de rotação** 4x4, que será usada para alinhar o sistema de coordenadas do objeto a desenhar com a curva.

De forma a facilitar a visualização e depuração das trajetórias animadas, também decidimos desenhar a **curva de Catmull-Rom** correspondente no ecrã:

```

glBegin(GL_LINE_LOOP);
for (float tt = 0; tt < 1; tt += 0.01f) {
    float p[3], d[3];
    getGlobalCatmullRomPoint(tt, transform.curvePoints, p, d);
    glVertex3f(p[0], p[1], p[2]);
}
glEnd();

```

3.2.1.2. Rotação

Além do novo tipo de translação, também implementamos a **rotação animada**, através da qual um objeto gira continuamente em torno de um eixo ao longo de um determinado intervalo de tempo.

Esta rotação é ativada sempre que o campo `time` está definido no XML, substituindo o uso do `angle` tradicional:

```

...
<rotate time="10" x="0" y="1" z="0" />
...

```

No código, esta rotação é tratada da seguinte forma:

```

// --- Rotação com tempo ---
else if (transform.rotate[0] == 0 && transform.time > 0) {
    float elapsed = glutGet(GLUT_ELAPSED_TIME) / 1000.0f;
    float angle = fmod((elapsed / transform.time) * 360.0f, 360.0f);
    glRotatef(angle, transform.rotate[1], transform.rotate[2], transform.rotate[3]);
}

```

No momento de aplicação deste tipo de transformação, primeiramente verificamos se é uma rotação animada. Se sim, o ângulo atual de rotação é então calculado com base no tempo decorrido (`elapsed`) e o tempo de rotação estipulado (`transform.time`), criando, assim, uma rotação contínua e cíclica.

4. Sistema Solar

Nesta fase, a cena do Sistema Solar foi atualizada de forma significativa, com o objetivo de tirar partido das novas funcionalidades introduzidas.

Para isso, o seu gerador foi alterado, permitindo que o sistema solar incluia:

- **Rotações animadas em torno do próprio eixo** foram aplicadas ao Sol, planetas, luas, asteroides e ao novo cometa, simulando o movimento de rotação de cada objeto;
- **Rotações circulares à volta do Sol**, implementadas através de translações com curvas Catmull-Rom a todos os grupos dos planetas com suas luas e de rotações animadas ao grupo de asteroides.
- **Um novo modelo de cometa**, que é gerado a partir de um ficheiro `.patch`, recorrendo à técnica de superfícies de Bézier. Ele possui também uma trajetória própria, definida por uma curva Catmull-Rom, a qual é animada de forma contínua e também gira em torno do Sol, acompanhando o dinamismo do resto do sistema;

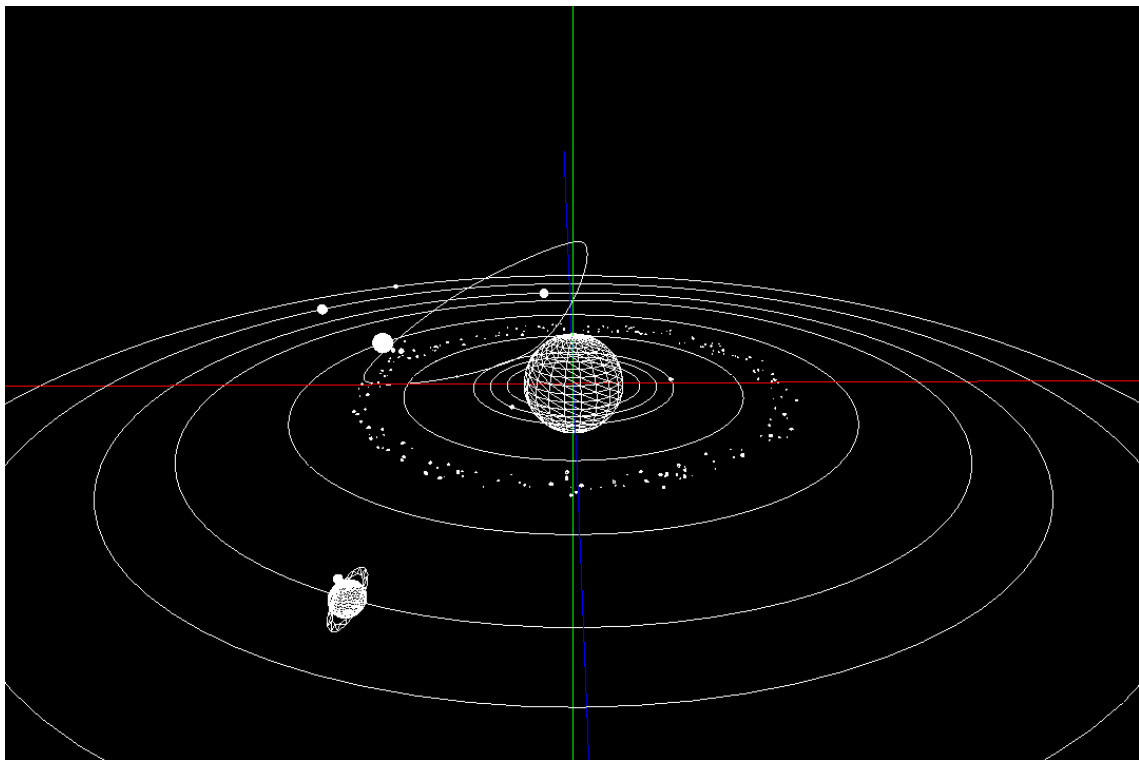


Figura 2: Representação da cena do Sistema Solar com rotações e translações animadas, incluindo o cometa

Também disponibilizamos um vídeo demonstrativo desta cena, onde é possível observar todas as funcionalidades implementadas em funcionamento, reforçando o realismo e complexidade do sistema animado. Este pode ser encontrado na diretoria:

/scenes/our_tests/test_files_phase_3_videos/solar_system.mp4

5. Testes

Para validar a implementação das novas funcionalidades introduzidas nesta fase — nomeadamente a geração de superfícies de Bézier e as transformações animadas — foram utilizados testes visuais.

As figuras seguintes ilustram dois testes realizados com o modelo clássico do *teapot* gerado com a nova :

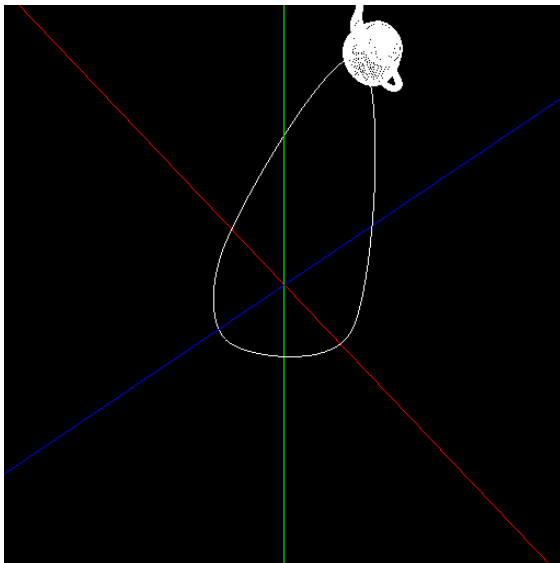


Figura 3: Teste 1 - Rotação animada de um *teapot* animado a seguir uma curva de Catmull-Rom com alinhamento ativado

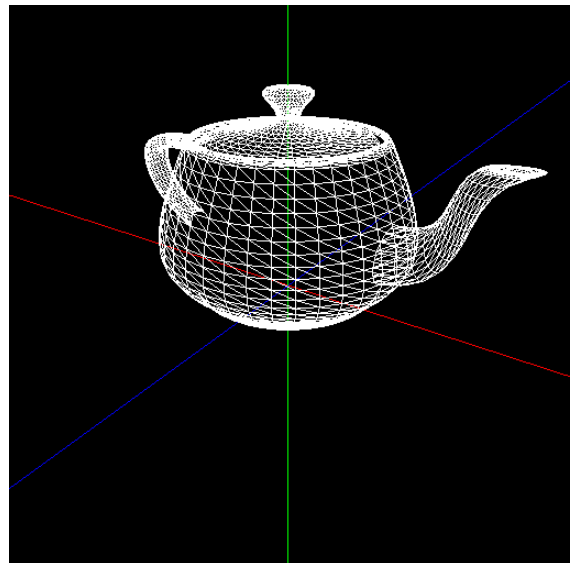


Figura 4: Teste 2 - *Teapot* estático

Estes testes asseguram que os objetos seguem corretamente as trajetórias definidas no ficheiro XML e que a orientação dinâmica corresponde adequadamente à derivada da curva, promovendo um comportamento coerente nas animações.

Além das imagens, também disponibilizamos um vídeo demonstrativo que permitem observar com maior clareza as animações implementadas. Este pode ser encontrado na diretoria:

/scenes/our_tests/test_files_phase_3_videos/teapot.mp4

6. Conclusões

Nesta fase, o projeto atingiu um novo nível de complexidade e realismo com a introdução de **movimentos animados**, novos tipos de modelos baseados em **superfícies de Bézier** e uma melhoria significativa na eficiência do motor gráfico com a adoção de **VBOs**.

Com todos os objetivos da terceira fase concluídos com sucesso, o projeto encontra-se agora preparado para a sua etapa final. Na próxima fase, o foco será a adição de iluminação, texturas e cores, mas, para além destes requisitos, pretendemos ainda enriquecer o projeto com ideias adicionais que tragam maior profundidade visual e interatividade à simulação, explorando o potencial criativo da aplicação.